

OmniCursor

Architecture, Build Plan & Hooks Integration

Status as of March 30, 2026 | Deadline: May 15, 2026 | 6.5 weeks remaining

Team: Julian Irusta, Samuel Simmons, Kailash Turimella, Andy Wu
Sponsor: Jonah Gray — OmniNode.ai
Repo: github.com/Julian0444/OmniCursor
Course: CS 490 — Senior Team Project, University of San Francisco

Current Status Snapshot

Metric	Current	Target (May 15)
MCP tools implemented	3 / 6	6 tools (+ hooks handling automatic behavior)
Skills implemented	5	15 (10 more from omniclaude)
Routing categories	5 + fallback	15 agents (YAML configs from omniclaude)
Cursor hooks	0 / 4	4 hooks: beforeSubmitPrompt, beforeShellExecution, afterFileEdit, stop
Tests passing	34	50+ including hook integration tests
Persistence layer	Not started	PostgreSQL with learned_patterns table

1. Executive Summary

OmniCursor is the Cursor-facing integration layer for OmniNode. It is not the whole OmniNode platform — it is the Cursor equivalent of omniclaude.

CRITICAL DISCOVERY: Cursor now has a hooks system (v1.7 beta) with 6 lifecycle events. This changes the entire architecture. OmniCursor can now implement automatic agent routing, compliance checking, and context injection — without the LLM needing to call MCP tools explicitly.

Before hooks (old approach): Rules + MCP tools only. The LLM had to “remember” to call `get_agent_context` and `check_compliance`. Routing was passive.

After hooks (new approach): Hooks fire automatically on every prompt, edit, and session end. Agent routing happens before the prompt reaches the model. Compliance checking runs after every file edit. The LLM gets enriched context without doing anything — this is the same architecture that makes omniclaude powerful.

2. Architecture

2.1 Three-Layer System

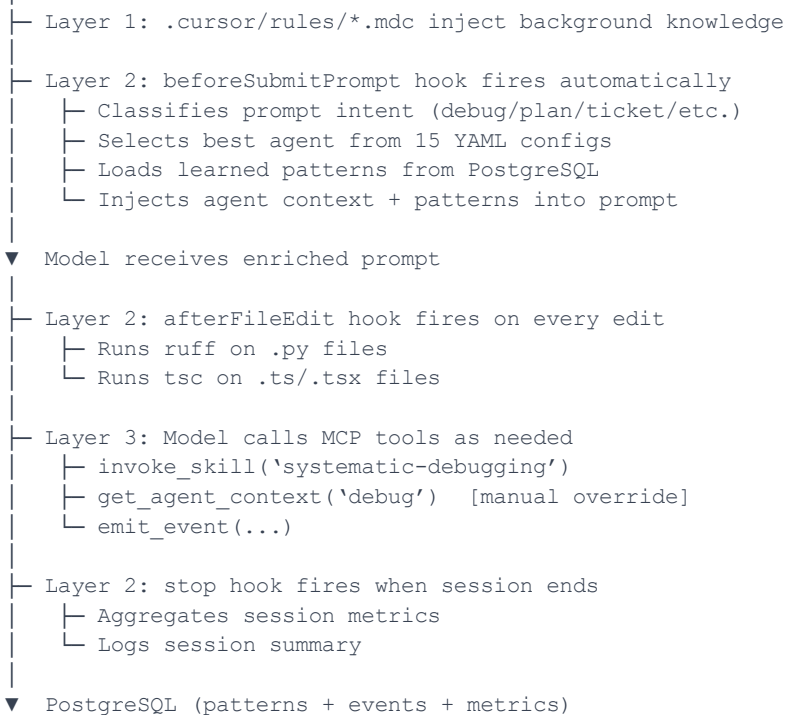
OmniCursor now operates on three layers, not two:

- **Layer 1 — Cursor Rules** (.cursor/rules/*.mdc) — Always-on knowledge base injected into every prompt. 7 rules covering OmniNode concepts, debugging, brainstorming, planning, tickets, and adapter stubs.
- **Layer 2 — Cursor Hooks** (.cursor/hooks.json) — Automatic scripts that fire at lifecycle events. Handle agent routing, compliance, shell safety, and session telemetry without LLM involvement.
- **Layer 3 — MCP Server** (src/omnicursor/) — Explicit tools the LLM can call on demand. Handle skill invocation, pattern queries, and manual overrides.

Key rule: Rules shape knowledge. Hooks shape behavior. MCP provides tools. All three layers complement each other.

2.2 High-Level System Flow

Developer types prompt in Cursor



2.3 Hook Mapping: Claude Code ↔ Cursor ↔ OmniCursor

Cursor Hook	Claude Code	Omnicleave Code	OmniCursor Implementation	Priority
beforeSubmit Prompt	UserPrompt Submit	agent_router.py (48KB), context_injection_wrapper.py	Classify prompt, select agent from 15 YAML configs, inject learned patterns	MUST HAVE
beforeShell Execution	PreToolUse (Bash)	bash_guard.py (24KB)	Validate shell commands against security policy (block rm -rf, DROP TABLE, force push)	SHOULD HAVE
beforeMCP Execution	PreToolUse (MCP)	workflow_guard.sh	Scope validation for MCP calls (light implementation)	NICE TO HAVE
beforeRead File	No equivalent	(none)	Context budget tracking, file-specific pattern injection	NICE TO HAVE
afterFile Edit	PostToolUse (Edit/Write)	post-tool-use-ruff.sh, post_tool_use_tsc_check.sh, pattern_enforcement.py	Auto-lint .py files with ruff, type-check .ts files with tsc, log edit events	MUST HAVE
stop	SessionEnd + Stop	session_outcome.py, metrics_aggregator.py	Aggregate session metrics, compute compliance score, log session summary	SHOULD HAVE

3. Hooks Configuration

The exact .cursor/hooks.json for OmniCursor:

```
{
  "version": 1,
  "hooks": {
    "beforeSubmitPrompt": [
      { "command": "python .cursor/hooks/on_prompt.py" }
    ],
    "beforeShellExecution": [
      { "command": "python .cursor/hooks/on_shell.py" }
    ],
    "afterFileEdit": [
      { "command": "python .cursor/hooks/on_edit.py" }
    ],
    "stop": [
      { "command": "python .cursor/hooks/on_stop.py" }
    ]
  }
}
```

Hook Handlers to Build

Handler	What It Does	Reuses from Omnicleave	Budget
on_prompt.py	Classifies prompt, selects agent from YAML configs, loads patterns from DB, injects context before prompt reaches model	agent_router.py (TriggerMatcher, ConfidenceScorer), context_injection_wrapper.py, 15 agent YAML configs	<500ms (matches omnicleave)

on_shell.py	Validates shell commands against security policy. Blocks dangerous ops (rm -rf, force push, DROP TABLE)	bash_guard.py (24KB) — nearly verbatim	<100ms
on_edit.py	Runs ruff on modified .py files. Runs tsc on modified .ts/.tsx files. Logs edit event.	post-tool-use-ruff.sh, post_tool_use_tsc_check.sh — nearly verbatim	<2s (linter time)
on_stop.py	Aggregates session metrics (files modified, tools used, duration). Computes compliance score. Logs session summary.	session_outcome.py, metrics_aggregator.py — simplified	<1s

4. MCP Tools (Updated)

With hooks handling automatic behavior, MCP tools focus on **explicitly invoked** capabilities:

Tool	Purpose	Phase	Status	Note
get_agent_context	Manual agent selection override	1	✓ Done	Hooks handle auto-routing; MCP is manual override
invoke_skill	Loads skill markdown by name	1	✓ Done	Core skill access — no change
check_compliance	Manual compliance check	2	✓ Done	Hooks handle auto-checks; MCP is manual override
store_pattern	Save learned pattern to DB	3	Next	Called by hooks or explicitly by model
get_patterns	Query pattern store	3	Next	Hooks inject patterns auto; MCP for debugging
emit_event	Log structured telemetry	3	Planned	Explicit event emission from skills

5. Skills Roadmap

22 of omnicaude’s 70+ skills are fully portable (methodology-based, no infrastructure dependencies). Current OmniCursor has 5. Target: 15 by demo.

Current (5 skills — done)

- systematic-debugging, brainstorming, writing-plans, plan-ticket, adapter-stub

Next 10 to port (from omnicaude, ordered by value)

#	Skill	What It Does	Effort	Source File
6	pr-review	PR assessment and merge readiness evaluation	Low	pr_review/SKILL.md
7	pr-polish	PR description refinement and quality improvement	Low	pr_polish/SKILL.md
8	hostile-reviewer	Adversarial code review for quality assurance	Low	hostile_reviewer/SKILL.md
9	defense-in-depth	Multi-layer validation and quality gates	Low	defense_in_depth/SKILL.md
10	merge-planner	Merge strategy planning and conflict resolution	Low	merge_planner/SKILL.md
11	insights-to-plan	Convert insights into actionable implementation plans	Low	insights_to_plan/SKILL.md
12	executing-plans	Plan execution orchestration	Low	executing_plans/SKILL.md
13	multi-agent	Parallel work orchestration (3 modes)	Medium	multi_agent/SKILL.md
14	handoff	Work handoff protocol between agents	Low	handoff/SKILL.md
15	using-git-worktrees	Git worktree methodology for isolated development	Low	using_git_worktrees/SKILL.md

6. Scope Lock

Category	Items
IN SCOPE	4 Cursor hooks (beforeSubmitPrompt, beforeShellExecution, afterFileEdit, stop). 6 MCP tools. 15 skills. PostgreSQL pattern storage. 15 agent YAML configs. Ruff + TSC auto-linting. Session telemetry. Recorded demo.
REDUCED	Full ONEX runtime parity. Kafka event pipeline. Qdrant semantic search. 53 agents. 70+ skills. Full omnidash integration.
OUT	beforeMCPExecution hook. beforeReadFile hook. Full telemetry dashboard. Modifying OmniNode core repos. Claude Code-specific skills (deploy-local-plugin, etc.).

7. Team Ownership

Person	Primary Ownership	Current Focus	Next Deliverable
Julian	MCP server + hooks + architecture	Hooks infrastructure (hooks.json, on_prompt.py)	beforeSubmitPrompt with agent routing working
Samuel	Runtime integration + agents	Port 15 agent YAML configs from omniclaude	Agent configs adapted and tested with routing
Kailash	Skills + persistence	Port 10 new skills from omniclaude	15 total skills + PostgreSQL pattern store
Andy	Testing + compliance + demo	afterFileEdit hook (ruff + tsc) + test suite	Auto-linting working + 50 tests + demo script

8. Execution Phases

Phases 1 and 2 are complete. The hooks discovery reshapes Phases 3-6.

Phase 1 — Foundation (Mar 28 – Apr 3) ✓ COMPLETE

Repo restructured. MCP server with `get_agent_context` + `invoke_skill`. First skill + rule. 34 tests.

Phase 2 — Skills & Compliance (Apr 4 – Apr 10) ✓ COMPLETE

5 skills, 5 routing categories, `check_compliance`, compliance registry, rules updated with MCP guidance.

Phase 3A — Hook Infrastructure (Apr 11 – Apr 20) ← NEXT

Goal

Get Cursor hooks working with real handlers. This is the biggest architectural upgrade.

Deliverables

- Create `.cursor/hooks.json` with 4 hooks (`beforeSubmitPrompt`, `beforeShellExecution`, `afterFileEdit`, `stop`)
- **on_prompt.py** — agent routing adapted from `omniclaude's agent_router.py`. Keyword matching against 15 agent YAML configs. Returns agent context as injected prompt.
- **on_edit.py** — auto-lint adapted from `omniclaude's post-tool-use-ruff.sh` and `tsc check`. Runs on every `.py` and `.ts` file edit.
- **on_stop.py** — session summary logging adapted from `session_outcome.py`.
- Port 15 agent YAML configs from `omniclaude-main/plugins/onex/agents/configs/` (simplified versions).
- Integration testing: verify hooks fire correctly in Cursor v1.7+.

Done when

A prompt in Cursor automatically gets routed to the correct agent without the user calling any MCP tool. File edits auto-trigger ruff linting.

Risk

Cursor hooks are beta — API may change. **Mitigation:** keep handlers simple and decoupled. If hooks break, fall back to MCP-only mode (already working).

Phase 3B — Skills Expansion + Persistence (Apr 21 – May 1)

Goal

Port 10 more skills from `omniclaude`. Add PostgreSQL persistence for patterns. Add context injection to the `beforeSubmitPrompt` hook.

Deliverables

- Port 10 new skills from `omniclaude` (`pr-review`, `pr-polish`, `hostile-reviewer`, `defense-in-depth`, `merge-planner`, `insights-to-plan`, `executing-plans`, `multi-agent`, `handoff`, `using-git-worktrees`).
- **on_shell.py** — bash guard adapted from `omniclaude's bash_guard.py` (24KB).
- PostgreSQL via Docker Compose with `learned_patterns` table.
- **store_pattern** and **get_patterns** MCP tools.
- Context injection in `on_prompt.py` — query patterns from PostgreSQL and inject into prompt.
- **emit_event** MCP tool for explicit telemetry.

Done when

15 total skills working. A pattern stored in one session is retrieved in the next. Shell commands are validated before execution.

Phase 4 — Eval, Demo & Submission (May 2 – May 15)

Goal

Prove the system works, package the demo, and submit.

Deliverables

- Automated test suite: 15-20 scripted prompts across all categories.
- Routing accuracy measurement (target: 80%+).
- Written demo script showing full loop: prompt → hook routing → skill → compliance → pattern storage → retrieval.
- Recorded demo video.
- QUICKSTART.md: clone to demo in <15 minutes.
- Feature freeze May 11. Bug fixes only after that.
- Final presentation assets.

Done when

Repository tagged v1.0. Demo recording is final. Project submitted to CS 490.

Phase Summary

Phase	Window	Key Deliverable	Status	Architecture After
1. Foundation	Mar 28–Apr 3	MCP skeleton + 1 skill + 2 tools	✓	Rules + MCP (2 tools)
2. Skills & Compliance	Apr 4–Apr 10	5 skills + check_compliance + 34 tests	✓	Rules + MCP (3 tools)
3A. Hook Infra	Apr 11–Apr 20	4 hooks + 15 agent configs + auto-linting	NEXT	Rules + Hooks + MCP
3B. Skills + Persistence	Apr 21–May 1	10 more skills + PostgreSQL + context injection	Pending	+ PostgreSQL + 15 skills
4. Eval + Demo + Submit	May 2–May 15	Test suite + demo + freeze + submit	Pending	Final: locked v1.0

9. Definition of Done (May 15)

MVP Done (current)

- ✓ 3 MCP tools working (get_agent_context, invoke_skill, check_compliance)
- ✓ 5 skills invocable and passing rubrics
- ✓ 5 routing categories + generalist fallback
- ✓ 34 tests passing

Final Project Done (May 15)

- ✓ 4 Cursor hooks firing automatically (beforeSubmitPrompt, beforeShellExecution, afterFileEdit, stop)
- ✓ 6 MCP tools total
- ✓ 15 skills invocable and rubric-tested
- ✓ 15 agent YAML configs with automatic routing via hooks
- ✓ Auto-linting on every .py/.ts file edit
- ✓ Patterns stored and retrieved across sessions (PostgreSQL)
- ✓ Repo cloneable and runnable in <15 minutes
- ✓ Recorded demo showing the full loop

10. Risks & Mitigations

Risk	Impact	Mitigation
Cursor hooks are beta — API may change	High	Keep handlers simple. If hooks break, MCP-only mode still works (Phases 1-2 are complete).
Hook performance adds latency	Medium	Budget: <500ms per hook. Test early. omniclaude uses same budget successfully.
PostgreSQL integration complexity	Medium	Start with file-based JSON stub, upgrade to PostgreSQL once flow works.
Routing accuracy below 80%	Medium	Iterate on agent YAML trigger patterns. Add more context_triggers.
Scope creep with 70+ available skills	Medium	Hard cap at 15 skills. Quality over quantity. Each must pass rubric.

11. Final Recommendation

The hooks discovery is the single biggest architectural upgrade since the project started. It moves OmniCursor from ‘MCP wrapper with rules’ to ‘true Cursor-native equivalent of omniclaude.’ The #1 priority is getting beforeSubmitPrompt working with real agent routing — that alone makes the demo 10x stronger. Everything else builds on top of that.